

Python for Cybersecurity

Complete Weekly Theory + Practical Notes

Beginner-friendly study PDF

This document collects the Python fundamentals, program flow, collections, functions, file handling, exception handling, modules, security-oriented libraries, AI-assisted development, and practical security exercises covered this week.

1. Weekly Roadmap

- Python Fundamentals — variables, data types, operators, input/output, type conversion, coding practices.
- Program Flow — conditions, loops, nested conditions, control statements, and problem-solving.
- Collections — lists, tuples, sets, dictionaries, and basic manipulation.
- Functions — defining functions, parameters, return values, scope, and reusable code.
- File Handling — reading/writing files, JSON, CSV concepts, and logs.
- Exception Handling — try/except, debugging, and reliable scripts.
- Modules — importing modules, creating modules, packages, and virtual environments.
- Security Libraries — requests, JSON, os, socket, and subprocess.
- AI-Assisted Development — explanation, debugging, optimization, security review, and validation.
- Practical Tasks — password checks, log analysis, JSON reading, reusable modules, SHA-256 file hashing, and simple security workflows.

2. Python Fundamentals — Theory

Variables

A variable is a name that refers to a value. Python creates the variable when you assign a value.

```
name = "Ali"  
age = 20  
score = 95.5
```

- name stores text.
- age stores an integer.
- score stores a decimal number.

Basic Data Types

- str — text, for example: "hello"
- int — whole numbers, for example: 25
- float — decimal numbers, for example: 10.5
- bool — True or False
- list — ordered, changeable collection
- tuple — ordered, usually immutable collection
- set — collection of unique values
- dict — key/value collection

Input and Output

```
name = input("Enter your name: ")  
print("Hello", name)
```

input() receives text from the user. print() displays information on the screen. Remember that input() normally returns a string.

Type Conversion

```
age = int(input("Enter age: "))  
price = float(input("Enter price: "))
```

```
text = str(100)
```

Use `int()`, `float()`, and `str()` when you need to convert values to another type.

Operators

- Arithmetic: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- Comparison: `==`, `!=`, `>`, `<`, `>=`, `<=`
- Logical: `and`, `or`, `not`
- Assignment: `=`, `+=`, `-=`, `*=`, `/=`
- Membership: `in`, `not in`

3. Program Flow — Theory + Practical

if / elif / else

```
marks = 75

if marks >= 90:
    print("Excellent")
elif marks >= 50:
    print("Pass")
else:
    print("Fail")
```

Python checks conditions from top to bottom. The first matching branch runs.

Loops

```
for i in range(5):
    print(i)

count = 0
while count < 3:
    print(count)
    count = count + 1
```

`for` is useful when iterating over a sequence or known range. `while` repeats while its condition remains `True`.

Control Statements

- `break` — stop the loop.
- `continue` — skip the current iteration.
- `pass` — do nothing; useful as a placeholder.

Practical: Even/Odd

```
number = int(input("Enter a number: "))

if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Practical: Largest Number

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

if a > b:
    print("Largest:", a)
else:
    print("Largest:", b)
```

Problem-Solving Workflow

- Understand the problem.
- Identify inputs and expected outputs.
- Break the task into small steps.
- Write simple code.
- Run the code with normal and unusual inputs.
- Read errors carefully and fix the root cause.
- Review the final code for clarity and security.

4. Collections — Theory + Practical

Lists

```
students = ["Ali", "Sara", "John"]

students.append("Maya")
students.remove("John")

print(students)
```

Lists are ordered and changeable. Common operations include `append()`, `remove()`, indexing, and slicing.

Tuples

```
coordinates = (10, 20)
print(coordinates[0])
```

Tuples are ordered collections that are normally used when the values should not be changed.

Sets

```
ips = {"10.0.0.1", "10.0.0.2", "10.0.0.1"}
print(ips)
```

Sets store unique values. They are useful when duplicate entries should be removed.

Dictionaries

```
user = {
    "name": "Ali",
    "status": "active"
}

print(user["name"])
```

A dictionary stores data using keys and values.

Practical: Student Data Manager

```
students = []

student = {
    "name": "Ali",
    "marks": 85
}

students.append(student)

for student in students:
    print(student["name"], student["marks"])
```

5. Functions — Theory + Practical

A function is a reusable block of code. It helps avoid repeating the same logic.

```
def check_number(number):
    if number % 2 == 0:
        return "Even"
    else:
        return "Odd"

result = check_number(10)
print(result)
```

- def starts a function definition.
- Parameters receive values.
- return sends a result back to the caller.
- A local variable normally exists inside the function.
- Reusable functions make security scripts easier to test and maintain.

Practical: Calculator

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

x = int(input("Enter first number: "))
y = int(input("Enter second number: "))

print("Add:", add(x, y))
print("Subtract:", subtract(x, y))
```

6. File Handling — Theory + Practical

Reading a Text File

```
with open("security.log", "r") as file:
    for line in file:
        print(line.strip())
```

with open(...) safely manages the file and closes it automatically after the block. "r" means read mode.

Writing a File

```
with open("report.txt", "w") as file:
    file.write("Security report")
```

"w" writes a file and can replace existing contents. Use "a" to append new content.

CSV and JSON

CSV is useful for table-like data. JSON is common for structured data exchanged by applications and APIs.

```
import json

data = {
    "name": "Ali",
    "status": "active"
}

with open("user.json", "w") as file:
    json.dump(data, file, indent=4)
```

Practical: Simple Log Checker

```

file = open("security.log", "r")

failed = 0

for line in file:
    if "FAILED" in line:
        failed = failed + 1

file.close()

print("Failed logins:", failed)

```

Practical: JSON Reader

```

import json

file = open("users.json", "r")
data = json.load(file)
file.close()

for user in data:
    print(user["name"])
    print(user["status"])

```

7. Exception Handling — Theory + Practical

Exceptions are runtime problems such as invalid input, missing files, or network failures. try/except lets a program handle expected failures instead of crashing unexpectedly.

```

try:
    number = int(input("Enter a number: "))
    print(10 / number)

except ValueError:
    print("Please enter a valid number")

except ZeroDivisionError:
    print("Cannot divide by zero")

```

- try — code that may fail.
- except — how to handle a particular error.
- else — runs when no exception occurs.
- finally — runs whether an exception occurred or not.

8. Modules, Packages, and Virtual Environments

Importing a Module

```

import math

print(math.sqrt(25))

```

A module is reusable Python code. Python includes many built-in modules, and third-party packages can be installed when needed.

Creating a Reusable Security Module

security.py

```

def check_password(password):
    if len(password) >= 8:
        return "Good"
    else:
        return "Too short"

```

main.py

```
from security import check_password

password = input("Enter password: ")
result = check_password(password)

print(result)
```

Windows Virtual Environment

```
python -m venv venv
venv\Scripts\activate
python -m pip install --upgrade pip
pip freeze
deactivate
```

A virtual environment keeps a project's packages isolated from other Python projects.

9. Security-Oriented Python Libraries

requests

requests is commonly used for HTTP requests. For reliable security scripts, use HTTPS where appropriate, timeouts, exception handling, and certificate verification.

```
import requests

response = requests.get(
    "https://example.com",
    timeout=5
)

print(response.status_code)
```

os

The os module provides operating-system related functionality such as paths and environment information.

```
import os

print(os.getcwd())
```

socket

The socket module provides low-level network communication. A TCP connection uses an IP/hostname and port.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.settimeout(5)
sock.connect(("example.com", 80))

print("Connection successful")

sock.close()
```

Mental model: socket() → settimeout() → connect() → send()/recv() → close(). A successful connect means a TCP connection was established; it does not mean an HTTP request was exchanged.

Safer Socket Error Handling

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    sock.settimeout(5)
    sock.connect(("example.com", 80))
```

```

        print("Connection successful")

except socket.timeout:
    print("Connection timed out")

except ConnectionRefusedError:
    print("Connection refused")

except socket.gaierror:
    print("Could not resolve hostname")

finally:
    sock.close()

```

subprocess

subprocess allows Python to run external programs on the local computer.

```

import subprocess

result = subprocess.run(
    ["ipconfig"],
    capture_output=True,
    text=True
)

print(result.stdout)

```

- `capture_output=True` captures normal output and error output.
- `text=True` returns strings instead of bytes.
- `result.stdout` is normal output.
- `result.stderr` is error output.
- `result.returncode` indicates success or failure.

Security: avoid `shell=True` when untrusted input can reach the command. Prefer an argument list such as `["ipconfig"]` rather than building a shell command from user input.

10. SHA-256 and File Integrity

SHA-256 is a cryptographic hash function. It converts data into a fixed-size digest. A small change to the input normally produces a very different digest.

```

import hashlib

with open("example.txt", "rb") as file:
    data = file.read()

hash_value = hashlib.sha256(data).hexdigest()

print(hash_value)

```

- Hashing is one-way in normal use; it is not encryption.
- The same file content produces the same SHA-256 digest.
- SHA-256 is useful for file integrity checks.
- SHA-256 alone is not appropriate for storing passwords. Password storage should use a password-hashing algorithm such as Argon2id, scrypt, or bcrypt.

11. Practical Log Analysis with Counter

```

from collections import Counter

failed_ips = []

```



```

with open("auth.log", "r") as file:

    for line in file:

        if "Failed login" in line:

            ip = line.split()[-1]

            failed_ips.append(ip)

counts = Counter(failed_ips)

for ip, count in counts.items():
    print(ip, "→", count, "failed attempts")

```

Line-by-Line Meaning

- Counter is imported from collections to count repeated values.
- failed_ips = [] creates an empty list.
- with open(..., "r") opens auth.log for reading and closes it automatically.
- for line in file reads the log one line at a time.
- if "Failed login" in line selects failed-login records.
- line.split() breaks a line into whitespace-separated words.
- [-1] selects the last word; this assumes the IP is the last field.
- append(ip) stores the IP in the list.
- Counter(failed_ips) counts how many times each IP appears.
- counts.items() provides each IP and its count.
- print() displays the final report.

Example Log

```

Failed login alice 10.0.0.5
Successful login bob 10.0.0.8
Failed login john 10.0.0.5
Failed login admin 10.0.0.10
Failed login test 10.0.0.5

```

Example Output

```

10.0.0.5 → 3 failed attempts
10.0.0.10 → 1 failed attempts

```

Limitation: real log formats can be more complicated, so taking the last word is only correct when the log format guarantees that the IP address is the final field.

12. AI-Assisted Development

AI can act as a teacher, debugger, reviewer, and programming assistant. The important rule is: do not copy, paste, and trust code blindly.

Good AI Workflow

- Ask AI to explain the theory and every important line.
- When debugging, provide the code, complete traceback, expected behavior, and actual behavior.
- Understand the root cause before accepting a fix.

- Ask for readability, error handling, and security improvements.
- Test normal, boundary, invalid, and edge cases.
- Review generated code for command injection, unsafe file handling, secrets exposure, weak authentication, and network security issues.
- Verify that the final code actually behaves as expected.

Unsafe Example

```
import subprocess

command = input("Enter command: ")
subprocess.run(command, shell=True)
```

This pattern can allow command injection because untrusted user input is passed to a shell. A safer design is to avoid arbitrary shell commands and use fixed commands with argument lists where possible.

Useful AI Prompt Structure

```
Context:
I am learning Python for cybersecurity.

Goal:
Explain and fix this script.

Code:
[paste code]

Problem:
[describe the error or unexpected result]

Please:
1. Explain the theory.
2. Explain each important line.
3. Identify the root cause.
4. Give the smallest safe fix.
5. Add security improvements.
6. Give test cases.
```

13. Beginner Cybersecurity Practical Projects

Project 1 — Password Length Checker

```
password = input("Enter password: ")

if len(password) >= 8:
    print("Password is okay")
else:
    print("Password is too short")
```

This is a learning example for conditions and string length. A real password policy should consider more than length and should never log the password.

Project 2 — Simple Security Report

```
failed = 0

file = open("security.log", "r")

for line in file:
    if "FAILED" in line:
        failed = failed + 1

file.close()

print("Security Report")
```

```

print("-----")
print("Failed logins:", failed)

if failed >= 3:
    print("Warning: Many failed logins")
else:
    print("Everything looks okay")

```

Project 3 — JSON User Reader

```

import json

with open("users.json", "r") as file:
    data = json.load(file)

for user in data:
    print(user["name"])
    print(user["status"])

```

Project 4 — File SHA-256 Utility

```

import hashlib

filename = input("Enter file name: ")

with open(filename, "rb") as file:
    data = file.read()

hash_value = hashlib.sha256(data).hexdigest()

print("SHA-256:", hash_value)

```

For very large files, reading in chunks is more memory-efficient; this simple version is intended for learning.

Project 5 — Simple Security Monitor

```

failed = 0

with open("security.log", "r") as file:
    for line in file:
        if "FAILED" in line:
            failed = failed + 1

print("Security Report")
print("-----")
print("Failed logins:", failed)

if failed >= 3:
    print("Warning: Many failed logins")
else:
    print("Everything looks okay")

```

14. Practice Tasks

- Write a program that checks whether a number is positive, negative, or zero.
- Write a loop that prints numbers from 1 to 10.
- Create a list of five IP addresses and print each address.
- Create a dictionary containing username and status.
- Write a function that checks whether a number is even.
- Read a log file and count lines containing ERROR.
- Read a JSON file and print all usernames.
- Create a module with a function that validates a username.

- Calculate the SHA-256 hash of a small test file.
- Modify the failed-login analyzer to show only IPs with at least 3 failures.
- Add exception handling to the log analyzer.
- Review a subprocess script and identify whether user input can reach a shell.

15. Security Checklist

- Never store passwords, API keys, access tokens, or private keys in source code.
- Do not print or log sensitive secrets.
- Use input validation.
- Avoid shell=True with untrusted input.
- Use HTTPS and sensible request timeouts for network requests.
- Use exception handling around operations that can fail.
- Use SHA-256 for integrity checks, not as a password-storage algorithm.
- Use secure random generation such as Python's secrets module for security tokens.
- Test scripts with invalid and unexpected input.
- Only perform network/security testing on systems you own or are explicitly authorized to test.

16. Quick Revision — Mental Models

- Variable = a name referring to a value.
- if = choose a path.
- for/while = repeat work.
- List = ordered changeable collection.
- Tuple = ordered collection intended to stay fixed.
- Set = unique values.
- Dictionary = key → value.
- Function = reusable block of logic.
- File = persistent data stored on disk.
- JSON = structured data format.
- Exception handling = controlled response to runtime problems.
- Module = reusable Python code.
- Socket = network communication endpoint.
- subprocess = Python starts another local program.
- SHA-256 = fixed-size cryptographic digest used for integrity verification.
- Counter = count repeated values.

17. End-of-Week Learning Workflow

Learn theory
↓
Write a small example

↓
Run the code
↓
Read the output
↓
Try invalid / edge cases
↓
Fix errors
↓
Review security
↓
Build a small project
↓
Explain the code in your own words

The goal is not only to make code run. The goal is to understand what the code does, why it works, how it can fail, and how to make it safer.

18. Summary

This week's work builds the foundation for Python-based cybersecurity automation. You practiced Python syntax and problem solving, learned collections and functions, worked with files and JSON, handled exceptions, created reusable modules, used sockets and subprocesses, learned SHA-256 for integrity, analyzed authentication logs, and learned how to use AI responsibly for development and security review.

Next recommended step: repeat the practical projects without looking at the solutions, then build one small tool from scratch using functions, file handling, exception handling, and a security-oriented library.